

Exercise 1 — Employee Database (JDBC + MySQL)

Database schema

- Database: `EmployeeDB`.
- Table `Employees`: `id` (INT, AUTO_INCREMENT, PRIMARY KEY), `name` (VARCHAR(50)), `age` (INT), `department` (VARCHAR(50)), `salary` (DOUBLE).

Employee class

- A plain model class mirroring the table: `id` (int), `name` (String), `age` (int), `department` (String), `salary` (double), with a constructor, getters/setters, and `toString()`.

Connecting

- Connect once at startup with `DriverManager.getConnection("jdbc:mysql://localhost:3306/EmployeeDB", "root", "")` (adjust the username/password to your local MySQL setup), keep the connection open for the whole session, and close it when the program exits (a try-with-resources on the connection, or a finally block, both work).
- If the connection attempt fails (wrong credentials, the database not yet created, the server not running), catch the exception and print a clear message rather than letting a raw stack trace through, and exit gracefully rather than continuing into a menu that has nothing to work with.

Menu

- Loop showing: 1) Add employee, 2) List all employees, 3) Update employee by id, 4) Delete employee by id, 5) Search by name or department, 6) Exit. Keep looping until Exit is chosen. On a non-numeric or out-of-range choice, print an error and re-show the menu rather than crashing.

Function behaviour

- Add: prompt for name, age, department, and salary from the keyboard, then insert with a `PreparedStatement` that binds all four values as parameters (not string concatenation) — this avoids SQL injection and correctly handles names containing quotes or special characters. Require age and salary to be positive, re-prompting on failure.
- List: `SELECT * FROM Employees`, print one row per line in a readable format.
- Update by id: prompt for the id (re-prompting if it isn't a valid number), then the new field values (validated the same way as Add), and run a parameterized `UPDATE ... WHERE id = ?`; print the number of affected rows — 0 means that id didn't exist.
- Delete by id: prompt for the id (re-prompting if it isn't a valid number), run a parameterized `DELETE FROM Employees WHERE id = ?`, and report success or failure based on the affected-row count.
- Search by name or department: prompt for a single search term, then run one query that checks both columns with wildcards on both sides, e.g. `WHERE name LIKE ? OR department LIKE ?` with the parameter bound as `"%" + term + "%"` for each placeholder — so the same search box matches either field. Print all matches, or a "No employees found" message if there are none.
- Catch `SQLException` around every database operation triggered from the menu and print a clear message rather than letting a raw stack trace through.

Exercise 2 — Product Database (JDBC + MySQL)

Database schema

- Database: `ProductDB`.
- Table `Products`: `id` (INT, AUTO_INCREMENT, PRIMARY KEY), `name` (VARCHAR(100)), `category` (VARCHAR(50)), `price` (DOUBLE), `quantity` (INT).

Product class

- A plain model class with the five fields (`id` as int, `name` and `category` as String, `price` as double, `quantity` as int), a constructor, getters/setters, and `toString()`.

Connecting

- Connect once at startup with `DriverManager.getConnection("jdbc:mysql://localhost:3306/ProductDB", "root", "")` (adjust the username/password to your local MySQL setup), keep the connection open for the whole session, and close it when the program exits (a try-with-resources on the connection, or a finally block, both work).
- If the connection attempt fails, catch the exception, print a clear message, and exit gracefully rather than continuing into a menu that has nothing to work with.

Menu and behaviour

- Same structure as a typical CRUD console app: 1) Add product, 2) List all products, 3) Update product by id, 4) Delete product by id, 5) Search by name or category, 6) Exit. On a non-numeric or out-of-range choice, print an error and re-show the menu rather than crashing.
- Implement each menu action as its own method (e.g. `addProduct`, `listProducts`, `updateProduct`, `deleteProduct`, `searchProducts`) that performs the JDBC work directly, called from the menu loop.
- Use `PreparedStatement` with bound parameters for every statement that includes user-supplied values (add, update, delete, search) — never build SQL by string concatenation.
- Add: prompt for name, category, price, and quantity; require price and quantity to be positive, re-prompting on failure.
- Update and delete by id: prompt for the id, re-prompting if it isn't a valid number; report success or failure based on the affected-row count (0 rows affected means that id didn't exist).
- Search by name or category: prompt for a single search term, then run one query that checks both columns with wildcards on both sides, e.g. `WHERE name LIKE ? OR category LIKE ?`, with the parameter bound as `"%" + term + "%"` for each placeholder.
- Catch `SQLException` around every database operation triggered from the menu and print a clear message rather than letting a raw stack trace through.

Hashtable requirement

- When listing all products (option 2), don't print directly from the `ResultSet`. Instead, load every row into a `Hashtable<Integer, Product>` first, keyed by `id`, and only then iterate the Hashtable's values to print the list. This is a deliberate requirement of the exercise (practicing `Hashtable`), even though a plain `List` would normally be the simpler choice here.

Bonus: the DAO pattern

- Once the program above is working, you can restructure it around the DAO (Data Access Object) pattern: declare a `ProductRepository` interface with method signatures for add, findAll, findById, update, delete, and search, then move your existing JDBC logic into a `ProductRepositoryImpl` class that implements it. Change the menu code to depend only on the `ProductRepository` type, never directly on JDBC calls or on `ProductRepositoryImpl` — that separation is the actual point of the pattern.
- Because each menu action is already its own method with one clear responsibility, this extension is mostly a matter of moving that existing logic behind the new interface and class, rather than rewriting it.

Exercise 3 — Employee Management Application (Design Pattern & Internationalization)

Database schema

- Database: `EmployeeDB`.
- Table `tblNhanVien`: `id` (INT, PRIMARY KEY), `name` (NVARCHAR(64)), `yearOfBirth` (INT), `address` (VARCHAR(256)), `phone` (VARCHAR(11)), `salary` (INT).

Employee class

- A plain model class mirroring the table: `id` (int), `name` (String), `yearOfBirth` (int), `address` (String), `phone` (String), `salary` (int), with a constructor, getters/setters, and `toString()`.

Menu

- Loop showing: 1) Add new employee, 2) List employees, 3) Update employee by id, 4) Delete employee by id, 5) Search employee, 6) Change language (Vietnamese ↔ English), 7) Exit. On a non-numeric or out-of-range choice, print an error and re-show the menu rather than crashing.

Function behaviour

- Add: prompt for id, name, yearOfBirth, address, phone, and salary. Since `id` isn't auto-generated in this table, reject an id that already exists and re-prompt; require `yearOfBirth` to be a plausible birth year (not in the future) and `salary` to be positive, re-prompting on failure.
- List: print every employee in a readable format.
- Update by id: prompt for the id (re-prompting if it isn't a valid number), then the new field values (validated the same way as Add), and update the matching row; report success or failure based on the affected-row count.
- Delete by id: prompt for the id (re-prompting if it isn't a valid number), delete the matching row, and report success or failure based on the affected-row count.
- Search employee: prompt for a name (or part of one) and print every employee whose name contains it, case-insensitively, or a "No employees found" message if there are none.
- Catch `SQLException` around every database operation triggered from the menu and print a clear message rather than letting a raw stack trace through.

Connection class (Singleton)

- Create a class, e.g. `DBConnection`, with a private static field holding the one instance, a private constructor that opens the JDBC `Connection`, and a public static `getInstance()` method that creates the instance the first time it's called and returns the same one on every later call — so the whole application shares exactly one `Connection` object. If the connection can't be opened, catch the exception and print a clear message rather than letting a raw stack trace through.

DAO pattern

- An `EmployeeDAO` interface declaring the CRUD/search method signatures, and an `EmployeeDAOImpl` class implementing them with JDBC, using `PreparedStatement` for every query that takes a parameter. The menu code should call only the interface, never the JDBC implementation directly.

Internationalization (Locale + ResourceBundle)

- Create two property files with the same set of keys, e.g. `messages_en.properties` and `messages_vi.properties` — one key per piece of UI text (menu items, prompts, success/error messages), e.g. `menu.add=Add new employee` in the English file, with the equivalent Vietnamese phrasing under the same key in the Vietnamese file.
- Load the active bundle with `ResourceBundle.getBundle("messages", locale)`, where `locale` is `Locale.ENGLISH` or a Vietnamese locale such as `new Locale("vi", "VN")`.
- Store the current `Locale` in a field on the main class. The "Change language" option toggles it and reloads the bundle. Every menu line and prompt should be read from the bundle rather than hard-coded, so the whole UI switches language immediately when that option is chosen. The selection only needs to last for the current run — it doesn't need to be saved between runs.

Exercise 4 — Advanced JDBC: Goods (Inventory) Management

Database schema

- Database: `InventoryDB`.
- Table `Products`: `productCode` (VARCHAR, PRIMARY KEY), `productName` (VARCHAR), `manufacturer` (VARCHAR), `quantity` (INT), `originalPrice` (DECIMAL, in VND), `vatRate` (DECIMAL, stored as a percentage, e.g. `10.00` for 10%).

InventoryItem class

- A plain model class mirroring the table: `productCode`, `productName`, `manufacturer` (String), `quantity` (int), `originalPrice` (double), `vatRate` (double), with a constructor, getters/setters, and `toString()`.

Connecting

- Connect once at startup with `DriverManager.getConnection("jdbc:mysql://localhost:3306/InventoryDB", "root", "")` (adjust the username/password to your local MySQL setup), keep the connection open for the whole session, and close it when the program exits.
- If the connection attempt fails, catch the exception, print a clear message, and exit gracefully rather than continuing into a menu that has nothing to work with.

Menu

- Loop showing: 1) Stock in, 2) Stock out, 3) Inventory, 4) Update, 5) Exit. On a non-numeric or out-of-range choice, print an error and re-show the menu rather than crashing.

Stock in — PreparedStatement + batch insert

- Prompt for how many line items are being received in this batch, then for each: product code, name, manufacturer, quantity, original price, VAT rate. Require `quantity` and `originalPrice` to be positive and `vatRate` to fall between 0 and 100, re-prompting on failure.
- Add each item as a parameterized `INSERT ... ON DUPLICATE KEY UPDATE quantity = quantity + VALUES(quantity)` call to a single `PreparedStatement` via `addBatch()` (the ON-DUPLICATE-KEY clause means re-stocking an existing product code adds to its quantity instead of erroring on the duplicate primary key), and execute the whole batch in one `executeBatch()` call once all items are entered — this is the batch-insert requirement.

Stock out — stored procedure

- Create a MySQL stored procedure, e.g. `sp_stock_out(IN p_code VARCHAR(50), IN p_qty INT)`, that first checks whether `p_code` exists at all — if not, it raises an error with `SIGNAL SQLSTATE '45000'` and a descriptive message. Otherwise, it checks the current quantity for `p_code`; if it's less than `p_qty`, it raises the same kind of error with its own descriptive message (either error cancels the transaction); if the quantity is sufficient, it decrements the quantity by `p_qty`.
- Call it from Java with a `CallableStatement` (e.g. `{call sp_stock_out(?, ?)}`); catch the `SQLException` raised by either signal and print a warning to the console — reflecting whichever of the two problems actually occurred — instead of letting a stack trace through.

Inventory and Update

- Inventory: list every product with all columns. No user input is involved in this query, so a plain `Statement` or an unparameterized `PreparedStatement` both work.
- Update: prompt for a product code, then new values for the editable fields — name, manufacturer, original price, VAT rate (not quantity, since quantity only ever changes via stock in/out), validated the same way as Stock in — and update the row with a parameterized `PreparedStatement`; print the number of affected rows, since 0 means that product code didn't exist.
- Catch `SQLException` around every database operation triggered from the menu (beyond the stock-out signal handling above) and print a clear message rather than letting a raw stack trace through.

Exercise 5 — Student Management Console Application (JDBC)

Build a student-management console application against a database named `qlsv`, with a table `tblSinhVien`.

Database schema (`tblSinhVien`)

- `id` — `INT(11)`, `AUTO_INCREMENT`, `PRIMARY KEY`, `NOT NULL`.
- `rollnumber` — `VARCHAR(8)`, `NOT NULL`.
- `name` — `VARCHAR(256)`, `NOT NULL`.
- `address` — `VARCHAR(512)`, `NULL` allowed.
- `phone` — `VARCHAR(11)`, `NULL` allowed.
- `gender` — `BIT(1)`, `NOT NULL`.

Student class

- A plain model class mirroring the table: `id` (`int`), `rollNumber` (`String`), `name` (`String`), `address` (`String`), `phoneNumber` (`String`), `gender` (`boolean`, `true` for male), with a constructor, getters/setters, and `toString()`.

Adding the MySQL JDBC driver

- Download the MySQL Connector/J jar (e.g. `mysql-connector-java-x.x.x-bin.jar`) and add it to your project's build path. In NetBeans: right-click the project → Properties → Libraries → Add JAR/Folder → select the driver jar. (For SQL Server instead of MySQL, add `sqljdbc4.jar` the same way.)
- Keep the jar inside your project (e.g. in a `libs` folder) rather than referencing it from an arbitrary location on disk, so the reference doesn't break if the project moves.

Connection details

- A locally installed MySQL server normally listens on `localhost`, port `3306`, with username `root` and an empty password by default — adjust the connection string if your setup differs.
- A locally installed SQL Server instance is normally addressed as `<machine-name>\<instance-name>` and is used the same way once you're connected.

Part 1 — Connect and Report Connection Status

- Connect to the `qlsv` database once at startup, keep the connection open for the whole session, and close it when the program exits. Print a message indicating whether the initial connection succeeded, and if it failed (e.g. because the database doesn't exist yet), catch the exception and print a clear message rather than letting a raw stack trace through.
- Before the `qlsv` database exists, connecting will fail with an "unknown database" error — that's expected. Create the `qlsv` database and the `tblSinhVien` table using the schema above, then run the program again: it should report a successful connection.

Part 2 — Insert Student Records from Keyboard Input

- Add a menu-driven loop: 1) Enter student information, 2) List of students, 3) Edit student, 4) Delete student, 5) Exit — looping until Exit is chosen, with an error and a re-shown menu on a non-numeric or out-of-range choice.
- Option 1 prompts, in order, for roll number, full name, address, phone number, and gender (entered as 1 for Male or 0 for Female — re-prompt on any other entry), then inserts a new row using a `PreparedStatement` with all five values bound as parameters — not string concatenation, so the insert can't be broken by a name or address containing a quote. Report "Added successfully" if the insert affected a row, or a failure message otherwise. Example entry: roll number `JF150801`, name Jang Hang Ra, address Seoul, phone `1234567890`, gender 0 (Female).

Part 3 — List Students

- Option 2 loads every row from `tblSinhVien` and prints one line per student between two rules of dashes, in the format `id|rollnumber| name - address - phone - gender`, with gender printed as "Male" or "Female". Continuing the example from Part 2:

```
1| C1505001| Ma Sieu - Tay Luong - 19001265 - Male
2| C1505002| Dieu Thuyen - To Chau - 19001001 - Female
3| JF150801| Jang Hang Ra - Seoul - 1234567890 - Female
```

Part 4 — Edit and Delete Student Records, by Roll Number

- Option 3 prompts for the roll number of the student to edit, then the same fields as Part 2's entry prompts (full name, address, phone number, gender), and updates the matching row using a parameterized `UPDATE ... WHERE rollnumber = ?`. Report "Updated successfully" if a row was affected, or a failure message otherwise — including when the roll number doesn't exist. Example: editing `JF150801` with new values Giau Hop Phat 1 / Motel / `69696969` / 1 (Male).
- Option 4 prompts for the roll number of the student to delete, and removes the matching row using a parameterized `DELETE FROM tblSinhVien WHERE rollnumber = ?`. Report "Deleted successfully" if a row was affected, or a failure message otherwise.
- Every statement in this exercise that includes a user-supplied value — insert, update, and delete alike — must use a `PreparedStatement` with bound (`?`) parameters; building any of these queries by concatenating strings together is not acceptable, since it both risks SQL injection and breaks on names or addresses containing a single quote.
- Catch `SQLException` around every database operation triggered from the menu and print a clear message rather than letting a raw stack trace through.